```
import re
from PySide6.QtGui import QSyntaxHighlighter, QTextCharFormat, QColor, QFont
from PySide6.QtCore import QRegularExpression

class LatexHighlighter(QSyntaxHighlighter):
    def __init__(self, parent=None, is_dark=False):
        super().__init__(parent)
        self.is_dark = is_dark
        self.refresh_rules() # Initialize rules based on the current theme

    def set_dark_mode(self, is_dark):
        self.is_dark = is_dark
        self.refresh_rules()
        self.rehighlight() # Force immediate redraw

    def refresh_rules(self):
        # TeXstudio-style color mapping
        if self.is_dark:
            # High-contrast colors for dark background
            self.colors = {
                "keyword": "#FF79C6",  # Pink
                "command": "#8BE9FD",  # Cyan
                "comment": "#6272A4",  # Muted Blue-Grey
                "math":    "#50FA7B",  # Green
                "brace":   "#FFB86C"   # Orange
            }
        else:
            # Classic TeXstudio colors for light background
            self.colors = {
                "keyword": "#800000",  # Dark Red
                "command": "#000080",  # Navy
                "comment": "#A0A0A0",  # Grey
                "math":    "#009300",  # Green
                "brace":   "#FF5500"   # Red-Orange
            }

        patterns = [
            (r"\\(?:begin|end|section|subsection|chapter)\b", "keyword"),
            (r"\\\w+", "command"),
            (r"(?<!\\)%.*", "comment"),  # Updated: ignores escaped % (\%)
            (r"\$.*?\$", "math"),
            (r"\{|\}", "brace")
            ]

        self.rules = []
        for pattern, key in patterns:
            fmt = QTextCharFormat()
            fmt.setForeground(QColor(self.colors[key]))
            if key == "keyword":
                fmt.setFontWeight(QFont.Bold)
            self.rules.append((re.compile(pattern), fmt))

    def create_format(self, style_key):
        cfg = self.styles.get(style_key, {})
        fmt = QTextCharFormat()

        if "color" in cfg: fmt.setForeground(QColor(cfg["color"]))
        if "bg" in cfg: fmt.setBackground(QColor(cfg["bg"]))
        if cfg.get("bold"): fmt.setFontWeight(QFont.Bold)
        if cfg.get("italic"): fmt.setFontItalic(True)

        return fmt

    def update_style(self, style_key, color_hex):
        # Update a specific category and refresh the view
        if style_key in self.styles:
            self.styles[style_key]["color"] = color_hex
            self.refresh_rules()
            self.rehighlight() # Force the editor to redraw

    def highlightBlock(self, text):
        for pattern, fmt in self.rules:
            for match in pattern.finditer(text):
                self.setFormat(match.start(), match.end() - match.start(), fmt)
```